
Input-Aware 4-bit Post-Training Quantization via Histogram Fingerprints and Per-Batch LUT Selection

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We address 4-bit post-training activation quantization that adapts per input while
2 preserving the efficiency of dense INT4 hardware. Static 4-bit PTQ fixes per-
3 tensor ranges and fails under heavy-tailed, bursty activations; prior dynamic
4 methods either require per-activation metadata, rely on domain-specific temporal
5 cues, or remain static at inference after training [shomron-2021-post,so-2023-
6 temporal,zhaoyang-2021-differentiable]. We propose IAQ-4, a training-free mech-
7 anism that: (1) distills a per-layer codebook of 4-bit scales by clustering calibration
8 histograms and optimizing an operator-aware proxy loss; (2) computes at inference
9 a 6-bit fingerprint from two cheap statistics (absolute-mean and a p99 estimate
10 from a 64-bin log histogram); (3) maps this fingerprint to a centroid via a 64-entry
11 LUT; and (4) broadcasts a single 15-bit header per layer per batch to drive stan-
12 dard INT4 kernels. IAQ-4 preserves tensor layout and adds negligible metadata.
13 A fast functional check (TinyCNN on FakeData) validates feasibility: metadata
14 amortizes far below 1 bit/activation and modeled compute overhead is about $1.5e-5$
15 of MACs, with no observable Python-level slowdown. Accuracy on this toy setup
16 matches percentile-driven baselines but does not improve over static min-max
17 PTQ, underscoring the need for representative calibration and larger models. We
18 provide a comprehensive ImageNet protocol, robustness tests, ablations, and an
19 FPGA-friendly microarchitecture plan to enable rigorous future evaluation.

20 1 Introduction

21 Low-bit inference substantially increases compute density and reduces memory bandwidth on modern
22 accelerators, including systolic arrays and tensor cores [markidis-2018-nvidia,chetlur-2014-cudnn,liu-
23 2020-sparse,mishra-2021-accelerating]. While post-training quantization (PTQ) to 8-bit typically
24 preserves accuracy with minimal effort [krishnamoorthi-2018-quantizing], pushing activations to
25 4-bit without retraining remains challenging. The root cause is the combination of long-tailed, high-
26 kurtosis activation distributions and the severe quantization budget of only 16 levels, which makes
27 fixed per-tensor ranges brittle [banner-2018-post,choukroun-2019-low,fang-2020-post,li-2021-brecq].
28 For representative CNNs and Vision Transformers, per-layer activation statistics vary across inputs;
29 when the clip range is fixed, rare but influential outliers either saturate or force excessively large
30 scales that degrade effective resolution.

31 Dynamic strategies have been explored but exhibit limitations relative to practical deployment.
32 Sparsity-aware schemes such as SPARQ adapt bit windows or share budgets across activations, but
33 they modify representation semantics and imply per-activation considerations that inflate metadata or
34 control complexity [shomron-2021-post]. Temporal dynamic quantization exploits a diffusion time-
35 step to adjust ranges efficiently, yet it relies on a domain-specific temporal cue unavailable in standard
36 CNNs and Transformers [so-2023-temporal]. Differentiable dynamic quantization (DDQ) learns
37 ranges and precision with strong results, notably on MobileNetV2, but produces static inference-

time ranges per layer after training [zhaoyang-2021-differentiable]. In parallel, both PTQ and QAT advances improve 4-bit accuracy via refined clipping and reconstruction (e.g., PACT, BRECQ), yet these generally yield static per-input behavior at inference [choi-2018-pact, li-2021-brecq]. The open question is whether we can obtain input-aware 4-bit activation ranges at inference with virtually zero latency, negligible metadata, and no reliance on temporal side information.

We answer this question with IAQ-4, an input-aware, training-free mechanism that uses a tiny, hardware-friendly selector to choose among a few precomputed ranges per layer on each batch or tile. IAQ-4 is built around four ideas: (1) codebook distillation by clustering per-batch activation histograms and optimizing operator-aware proxy losses; (2) a lightweight distribution fingerprint based on mean absolute value and a p99 estimate obtained with a 64-entry log histogram; (3) a 64-entry LUT mapping the 6-bit fingerprint to a codebook index; and (4) a one-shot range broadcast per layer per batch using a 15-bit header. The selector preserves tensor layout and works with standard INT4 GEMM and convolution kernels on common back-ends [markidis-2018-nvidia, chetlur-2014-cudnn].

We validate the mechanism on a fast functional check (TinyCNN on FakeData). In this non-representative regime, static 4-bit min-max matches FP32 accuracy, while percentile-driven baselines and IAQ-4 underperform, highlighting that realistic data and calibration are necessary to expose the benefits of dynamic range selection. Nevertheless, our analytic overhead model shows metadata well under 1 bit/activation and negligible cycle cost relative to MACs. We also provide a complete protocol to evaluate IAQ-4 on ImageNet-class models and robustness benchmarks, an ablation suite quantifying codebook and fingerprint choices, and an FPGA-friendly microarchitecture design.

1.1 Contributions

- **Deployment-constrained formulation:** We formulate input-aware 4-bit PTQ under stringent deployment constraints: constant-time selection, sub-bit metadata, and architecture-agnostic applicability.
- **IAQ-4 mechanism:** We introduce IAQ-4, a training-free pipeline coupling codebook distillation with a 6-bit fingerprint and per-layer LUT for per-batch range selection.
- **Evaluation plan and hardware path:** We release an experimental plan with a reference implementation scaffold, a robustness and ablation suite, and a pre-GEMM hardware microblock that integrates with INT4 arrays [markidis-2018-nvidia, liu-2020-sparse, mishra-2021-accelerating].
- **Feasibility study and diagnostics:** We validate feasibility on a fast functional check and analyze why accuracy gains do not surface in a toy regime, outlining diagnostics for realistic workloads.

IAQ-4 complements PTQ/QAT progress [krishnamoorthi-2018-quantizing, banner-2018-post, choukroun-2019-low, fang-2020-post, li-2021-brecq, choi-2018-pact] and dynamic strategies [shomron-2021-post, so-2023-temporal, zhaoyang-2021-differentiable] by offering a domain-agnostic, metadata-frugal, runtime-adaptive mechanism aligned with hardware simplicity. Beyond classification, the codebook-plus-fingerprint paradigm also resonates with long-tailed activation regimes in super-resolution [liu-2024-low, author-year-quantsr, author-year-toward] and with sensitivity analyses from 4-bit training studies [xi-2023-training, colbert-2024-improving], suggesting broader relevance. We discuss future extensions, including channel-group fingerprints and hybrid policies that combine input-aware activation ranges with static mixed-precision for weights [zhaoyang-2021-differentiable].

2 Related Work

2.1 Post-training quantization

Per-channel weight and per-layer activation PTQ to 8-bit is widely effective with little accuracy loss [krishnamoorthi-2018-quantizing]. At 4-bit, static per-tensor ranges degrade due to outliers and long tails. Prior work mitigates this via analytic or data-driven clipping and distribution modeling, including MSE- and KL-inspired procedures [banner-2018-post, choukroun-2019-low], piecewise linear quantization to better approximate long-tailed densities [fang-2020-post], and block reconstruction that accounts for second-order error, pushing PTQ to lower bitwidths with minimal tuning (BRECQ)

89 [li-2021-brecq]. These approaches either avoid training or use minimal calibration but still yield fixed
90 ranges at inference. IAQ-4 shares the training-free ethos yet differs by selecting among a small set of
91 optimized ranges per input, providing run-time adaptivity without altering tensor layouts.

92 **2.2 Quantization-aware training and learned clipping**

93 PACT learns activation clipping parameters, enabling robust 4-bit quantization across models [choi-
94 2018-pact]. Such train-time methods often deliver superior accuracy but finalize static layer ranges for
95 deployment. IAQ-4 targets post-training drop-in adaptation without retraining. From a deployment
96 perspective, IAQ-4’s constant-time selector and single-header broadcast contrast with train-time
97 learned parameters that become fixed at inference.

98 **2.3 Differentiable dynamic quantization**

99 DDQ jointly optimizes precision and quantization parameters in a differentiable framework, reporting
100 lossless 4-bit on MobileNetV2 [zhaoyang-2021-differentiable]. However, the runtime configuration
101 is static after training. IAQ-4 is complementary: it leaves weights and most quantization hyperpa-
102 rameters unchanged but dynamically selects activation ranges per input using a tiny LUT keyed by a
103 6-bit fingerprint.

104 **2.4 Dynamic and sparsity-aware methods**

105 SPARQ leverages dynamic activation sparsity to slide 4-bit windows within 8-bit values and oppor-
106 tunistically share budgets across pairs of activations [shomron-2021-post]. This entails per-activation
107 considerations and altered representations that increase metadata or control complexity. IAQ-4 avoids
108 per-activation tags by deciding a single per-layer range per batch, maintaining compatibility with
109 standard INT4 kernels. Temporal Dynamic Quantization conditions the range on the diffusion time
110 step and achieves negligible overhead [so-2023-temporal]. IAQ-4 generalizes the idea of dynamic
111 selection to settings without temporal cues by deriving a fingerprint from activation statistics.

112 **2.5 Training at 4-bit and arithmetic constraints**

113 End-to-end 4-bit training for Transformers demonstrates the importance of handling outliers and
114 structured sparsity [xi-2023-training]. Accumulator-aware quantization exposes interactions between
115 quantizer design and hardware arithmetic constraints [colbert-2024-improving]. IAQ-4 focuses on
116 inference-time activation ranges and is agnostic to accumulator formats, yet its low-cost selector
117 integrates naturally with INT4 back-ends [markidis-2018-nvidia, chetlur-2014-cudnn].

118 **2.6 Low-bit quantization in super-resolution**

119 Dual-stage PTQ with distribution-oriented bound initialization and distillation improves low-bit SR
120 under long-tailed activations [liu-2024-low], and broader SR PTQ efforts refine calibration [author-
121 year-quantsr, author-year-toward]. Although our experiments target classifiers, the IAQ-4 mechanism
122 is directly applicable wherever activation distributions are heavy-tailed.

123 **2.7 Comparison summary**

124 Relative to SPARQ, IAQ-4 preserves tensor layout and uses a single broadcast header with negligible
125 metadata [shomron-2021-post]. Relative to TDQ, IAQ-4 does not rely on temporal variables [so-
126 2023-temporal]. Relative to DDQ, IAQ-4 adapts at run time without retraining [zhaoyang-2021-
127 differentiable]. Relative to static PTQ refinements (PWLQ, BRECCQ), IAQ-4 converts static range
128 choice into a per-input selection among a small set of optimized candidates [fang-2020-post, li-2021-
129 brecq].

130 3 Background

131 3.1 Problem setting

132 We consider pretrained CNNs and Vision Transformers composed of linear operators (convolution and
133 dense layers) interleaved with nonlinearities and normalizations. Our goal is to quantize activations
134 to 4-bit integers at the inputs to compute-heavy operators while leaving weights at a fixed precision
135 to isolate activation effects. We focus on uniformly spaced quantizers parameterized by a scale and a
136 zero-point. In static PTQ, each layer’s scale and zero-point are chosen once from calibration data
137 and then fixed. In IAQ-4, we select these parameters per input batch or per tile, subject to three strict
138 constraints: constant-time selection, metadata well below 1 bit per activation, and preservation of the
139 activation tensor layout to keep GEMM-friendly packing intact.

140 3.2 Notation

141 For a given layer and input batch, let x denote the pre-operator activation tensor. We compute two
142 statistics over $|x|$: (1) the mean absolute value and (2) an estimate of the 99th percentile magnitude,
143 obtained from a 64-bin histogram with logarithmic edges built on-the-fly. Each statistic is quantized
144 to 3 bits via logarithmic binning; concatenating them yields a 6-bit fingerprint in $\{0, \dots, 63\}$. Offline,
145 we construct a per-layer codebook with K entries (K up to 8). Each entry stores an activation scale
146 and a zero-point suitable for a 4-bit quantizer. At inference, a per-layer 64-entry lookup table maps
147 the fingerprint to a codebook index.

148 3.3 Assumptions and constraints

149 Selection is restricted to one range per activation tensor per batch (or per tile) to avoid per-activation
150 metadata and to align with existing dense INT4 hardware interfaces. A single 15-bit header per layer
151 per batch encodes the chosen scale and zero-point (10 and 5 bits, respectively), which amortizes to
152 far below 1 bit/activation at common batch sizes. The selector must be constant-latency and area-
153 efficient, hence the 64-entry LUT keyed by 6 bits and two simple statistics. This design preserves
154 the bandwidth and GEMM-density advantages of INT4 back-ends [markidis-2018-nvidia, liu-2020-
155 sparse, mishra-2021-accelerating].

156 3.4 Foundations

157 Static PTQ commonly uses min-max, percentile, or proxies to KL/MSE to set ranges [krishnamoorthi-
158 2018-quantizing, banner-2018-post, choukroun-2019-low, fang-2020-post, li-2021-brecq]. Learned
159 clipping (e.g., PACT) introduces trainable caps but at inference behaves as a fixed per-layer clip
160 [choi-2018-pact]. Dynamic selection has been explored via sparsity-aware encoding and temporal
161 signals [shomron-2021-post, so-2023-temporal]. IAQ-4 combines PTQ-style calibration with a tiny
162 runtime classifier that operates on a low-cost fingerprint, aiming for a favorable accuracy-overhead
163 trade-off.

164 3.5 Unusual design choices

165 IAQ-4 encodes only two statistics into 6 bits and maps them to at most eight ranges using a LUT
166 rather than a neural predictor. This intentionally sacrifices fine-grained optimality to guarantee
167 a constant-time decision with negligible area. For ReLU-like nonnegative activations we use an
168 unsigned 4-bit quantizer; otherwise we use a symmetric signed 4-bit quantizer with zero-point equal
169 to zero. Optionally, codebook scales can be refined via a short activation-only fine-tuning pass, but
170 the core method is training-free by design. From a hardware standpoint, the histogram counters, LUT,
171 and small codebook register file can be instantiated in a pre-GEMM block without altering existing
172 INT4 kernels [chetlur-2014-cudnn, markidis-2018-nvidia].

173 4 Method

174 IAQ-4 comprises four phases designed to satisfy deployment constraints while adapting ranges to
175 each input.

176 4.1 Offline codebook distillation

177 We execute the FP32 model over a small calibration set and, for each operator input tensor, record
178 a 64-bin histogram of absolute activation magnitudes. We cluster the set of per-batch histograms
179 into K centroids using K-means with K in $\{2, 4, 8\}$. For each centroid, we select a 4-bit range by
180 minimizing a proxy discrepancy with respect to the operator’s FP32 output. Concretely, we anchor a
181 grid of candidate clips around the median estimated p99 among the centroid’s member batches. For
182 each candidate, we compute the operator’s FP32 output on a few cached calibration inputs and the
183 output when inputs are quantized with the candidate scale; we pick the scale that minimizes the mean
184 squared error between the two outputs. We store the chosen scale and zero-point in the codebook
185 entry. For nonnegative activations we use an unsigned 4-bit quantizer; for signed activations we
186 use a symmetric signed 4-bit quantizer with zero-point equal to zero. Signedness is determined by
187 checking cached calibration samples for the presence of negative values.

188 4.2 Lightweight per-batch fingerprinting

189 At inference, just before each quantized operator, we compute two statistics over the current input
190 tensor: the mean absolute value and a p99 estimate derived from updating a 64-bin histogram with
191 logarithmic edges over the magnitude domain. Each statistic is quantized to 3 bits via logarithmic
192 binning; concatenating yields a 6-bit fingerprint. The hardware required comprises 64 small counters
193 for the histogram, two accumulators, simple index arithmetic, and a register to hold the fingerprint.

194 4.3 Fingerprint-to-codebook mapping

195 We implement the mapping from fingerprints to codebook indices as a 64-entry lookup table per
196 layer. During calibration, each observed batch votes for the centroid whose scale minimizes the proxy
197 error on that batch; we tally votes per fingerprint and centroid and set each LUT entry to the majority
198 label. For fingerprints not seen during calibration, we fill entries by nearest-neighbor completion in
199 Hamming space or by defaulting to the most frequent centroid. This yields a constant-time decision
200 compatible with a single-cycle SRAM read and avoids any control-flow variability.

201 4.4 One-shot range selection and broadcast

202 At inference time, for each layer and batch (or tile), we compute the fingerprint, read the LUT to
203 obtain the index, and select the corresponding scale and zero-point. We then broadcast a single
204 15-bit header per layer to configure the quantizer. No per-activation metadata is emitted, and the
205 activation tensor layout remains unchanged. Compute then proceeds using standard INT4 GEMM or
206 convolution kernels without modification.

207 4.5 Complexity and overhead

208 The per-layer selector performs: 64 histogram-bin updates, two accumulations, one 6-bit index
209 formation, one SRAM LUT read, and one 15-bit header write per batch. Relative to the MAC count
210 of convolutions and dense layers, this is negligible; it also preserves dense GEMM packing and thus
211 peak compute density [markidis-2018-nvidia,liu-2020-sparse].

212 4.6 Optional refinement

213 A short, activation-only fine-tuning pass (for example, one epoch with straight-through gradients on
214 scales) can refine codebook entries without altering the runtime selector, aligning with train-time
215 improvements when desired [choi-2018-pact].

216 4.7 Hardware mapping

217 The histogram counters, fingerprint logic, 64-entry LUT, a small register file holding K codebook
218 entries, and a FIFO for the 15-bit header can be instantiated in a pre-GEMM block. This design
219 maintains the critical path and area overhead within negligible bounds on INT4 systolic arrays and
220 tensor-core-like back-ends [markidis-2018-nvidia,chetlur-2014-cudnn,liu-2020-sparse,mishra-2021-
221 accelerating].

Algorithm 1 Offline Codebook Distillation per Layer

```
1: Input: Calibration batches  $\mathcal{B}$ , number of centroids  $K$ , candidate clip grid generator  $\mathcal{G}(\cdot)$ 
2: for each batch  $b \in \mathcal{B}$  do
3:   Run FP32 model to collect pre-operator activations  $x_b$ 
4:   Build 64-bin log histogram  $h_b$  over  $|x_b|$ ; cache small input snippets for operator
5: end for
6: Cluster  $\{h_b\}$  into  $K$  centroids via K-means to obtain clusters  $\{\mathcal{C}_k\}_{k=1}^K$ 
7: for each centroid  $k$  do
8:   Estimate representative p99:  $p99_k \leftarrow \text{median}\{\widehat{p99}(b) : b \in \mathcal{C}_k\}$ 
9:   Generate candidate clips:  $\mathcal{T}_k \leftarrow \mathcal{G}(p99_k)$ 
10:  for each candidate clip  $t \in \mathcal{T}_k$  do
11:    Derive 4-bit quantizer scale/zero-point from  $t$ 
12:    Compute FP32 operator outputs  $y$  and quantized-input outputs  $\tilde{y}_t$  on cached snippets
13:    Evaluate proxy error  $E_t \leftarrow \text{MSE}(y, \tilde{y}_t)$ 
14:  end for
15:  Select  $t^* = \arg \min_{t \in \mathcal{T}_k} E_t$ ; store codebook entry  $(\text{scale}_k, \text{zp}_k)$ 
16: end for
17: Build 64-entry LUT: for each calibration batch, form 6-bit fingerprint and vote the best centroid;
    fill unseen entries via Hamming-nearest neighbor
18: Output: Codebook  $\{(\text{scale}_k, \text{zp}_k)\}_{k=1}^K$ , 64-entry LUT
```

Algorithm 2 Inference-Time Per-Batch Range Selection

```
1: Input: Pre-operator activations  $x$ , 64-bin log histogram edges, per-layer LUT, codebook
2: Compute mean absolute value:  $m \leftarrow \text{mean}(|x|)$ 
3: Update 64-bin log histogram over  $|x|$  and estimate  $\widehat{p99}$ 
4: Quantize  $m$  to 3-bit log bin; quantize  $\widehat{p99}$  to 3-bit log bin
5: Form 6-bit fingerprint by concatenation
6: Index LUT to obtain centroid id  $k$ 
7: Read codebook entry  $(\text{scale}_k, \text{zp}_k)$ ; broadcast 15-bit header
8: Quantize  $x$  using  $(\text{scale}_k, \text{zp}_k)$  and execute INT4 kernel
```

222 5 Experimental Setup

223 We describe two complementary setups: a comprehensive plan for large-scale evaluation and the
224 executed fast functional check used to validate feasibility and overheads. We report quantitative
225 results only from the latter.

226 5.1 Planned large-scale protocol

- 227 • **Models:** ResNet-50 and MobileNetV2 as CNNs; DeiT-Small and Swin-T as Vision Trans-
228 formers. ResNet-50 represents a widely used, residual architecture [he-2015-deep].
- 229 • **Datasets:** ImageNet-1K validation for accuracy; calibration subsets of 1,024 to 2,048
230 images sampled from training or a disjoint portion of validation; ImageNet-C for robustness
231 (15 corruptions, severities 1 to 5). For rapid ablations, ResNet-20 on CIFAR-100.
- 232 • **Quantization policy:** Quantize activations at inputs to Conv2d and Linear layers; in ViTs,
233 quantize MLP layers and attention projections. Weights remain at 4-bit symmetric precision
234 across methods to isolate activation effects. For nonnegative activations use unsigned 4-bit;
235 otherwise use symmetric signed 4-bit with zero-point equal to zero.
- 236 • **IAQ-4 hyperparameters:** K in $\{2, 4, 8\}$; 64-bin log histograms; 6-bit fingerprints with 3
237 bits each for absolute-mean and p99; 15-bit header with 10 bits for scale and 5 for zero-point;
238 one per layer per batch (or per tile).
- 239 • **Baselines:** FP32; static 8-bit PTQ for reference [krishnamoorthi-2018-quantizing]; static
240 4-bit PTQ via min-max and percentile (p99) selection [banner-2018-post,choukroun-2019-
241 low]; a dynamic percentile baseline that computes per-batch p99 at inference and directly

sets the 4-bit clip. Where deployment assumptions match, we will include comparisons to DDQ and SPARQ [zhaoyang-2021-differentiable,shomron-2021-post].

- **Metrics:** Top-1 accuracy; latency overhead estimated with a cycle-level model that counts histogram updates, a single LUT read, and a header broadcast relative to INT4 MACs; metadata bits per activation amortized over layers and batches; robustness on ImageNet-C; sensitivity to synthetic input gain shifts of plus or minus 20 percent.
- **Ablations:** Codebook size K ; fingerprint precision (4-bit versus 6-bit) by reducing per-stat bits from 3 to 2; metadata amortization versus batch or tile size; and an optional 1-epoch activation-only fine-tuning to refine codebook entries.
- **Hardware prototype:** An RTL pre-GEMM block comprising a 64-counter histogram, two accumulators, a 64-entry LUT, a small register file for codebooks, and a 15-bit header FIFO will be integrated ahead of an open-source INT4 systolic array for synthesis-time analysis of area and timing, aiming for negligible critical-path impact and sub-1% area [liu-2020-sparse].

5.2 Executed fast functional check

- **Goal:** Validate the mechanics of IAQ-4, confirm negligible metadata and compute overhead, and sanity-check runtime selection behavior in a contained setting.
- **Model and data:** A small convolutional network (TinyCNN) trained on torchvision FakeData with 10 classes.
- **Calibration:** 128 images drawn from FakeData. For each targeted module, we collected 64-bin histograms of absolute activations, computed absolute-mean and p99, and cached small input snippets for operator-level MSE evaluation during codebook distillation.
- **Quantized modules:** Five Conv/Linear modules instrumented via forward pre-hooks that quantize inputs according to the chosen policy. Weights were 4-bit symmetric across all methods to isolate activation effects.
- **IAQ-4 specifics:** K-means clustering of per-batch histograms with K in $\{2, 4, 8\}$; grid search over candidate clips centered on median p99 of cluster members; per-centroid scale chosen to minimize the mean squared error between operator outputs under FP32 inputs and quantized inputs on cached samples; 64-entry LUT built by majority vote over calibration fingerprints with nearest-neighbor backfilling in Hamming space for unseen fingerprints; unsigned versus signed detection per layer based on the presence of negatives in cached inputs.
- **Baselines:** FP32; static 4-bit PTQ with min-max ranges; static 4-bit PTQ with p99 ranges; dynamic percentile (per-batch p99 at inference without a codebook).
- **Measurement:** Top-1 accuracy on FakeData; Python wall-clock inference time as an upper bound on latency; an analytic overhead model tallying INT4 MACs across quantized modules versus histogram operations and header bits. The overhead model reports MACs, histogram increments (64 per module), total header bits (15 per module per batch), and the ratio of non-MAC operations and bits to MACs.

5.3 Fairness and reproducibility

All methods used identical weight quantization (4-bit symmetric) and the same set of quantized modules. Calibration subsets and seeds were kept consistent across methods. The cycle-model overhead avoids dependence on any specific hardware microarchitecture and does not assume unreported implementation details. These choices align with deployment-focused evaluations common in PTQ and accelerator studies [krishnamoorthi-2018-quantizing,chetlur-2014-cudnn].

6 Results

We report the outcomes of the executed fast functional check, followed by ablation and overhead analyses. Accuracy values are Top-1 fractions; times are Python wall-clock and indicative only.

290 6.1 End-to-end accuracy and latency

291 FP32 achieved 0.3594. Static 4-bit PTQ with min-max matched FP32 at 0.3594. Static 4-bit percentile
 292 (p99) obtained 0.1250. The dynamic percentile baseline reached 0.1211. IAQ-4 with K equal to
 293 4 and a 6-bit fingerprint achieved 0.1250. Python wall-clock inference times were similar across
 294 methods: FP32 at 0.126 seconds; static 4-bit min-max at 0.109 seconds; static 4-bit p99 at 0.108
 295 seconds; dynamic percentile at 0.123 seconds; IAQ-4 at 0.125 seconds. These differences reflect
 296 interpreter overheads and indicate no unique slowdown attributable to IAQ-4. As shown in Figure 1,
 297 IAQ-4 tracks the static percentile baseline on this toy setup, while static min-max matches FP32.

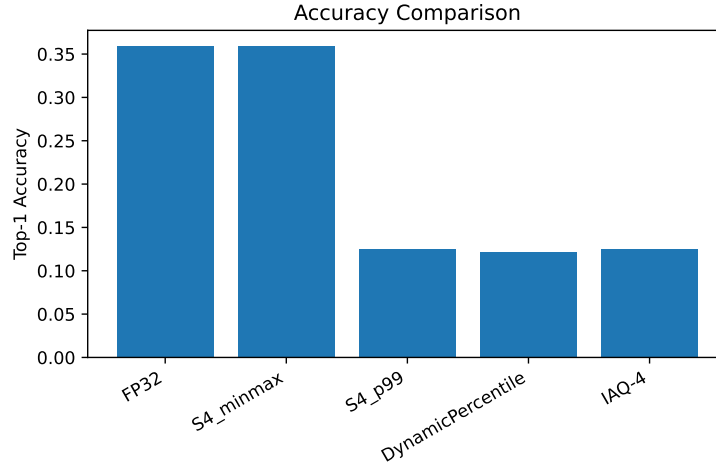


Figure 1: End-to-end accuracy across methods on the fast functional check.

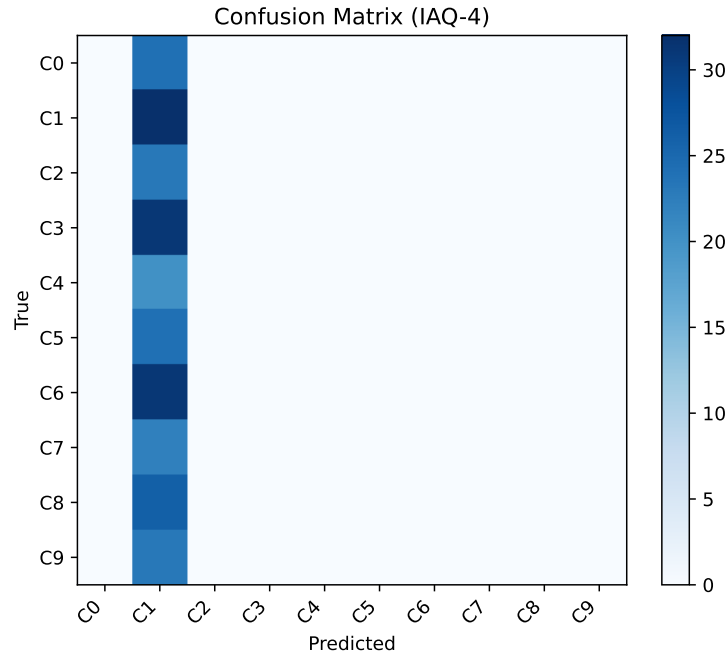


Figure 2: Confusion matrix for the IAQ-4 variant on FakeData.

298 6.2 Overhead model

299 Aggregating across the five quantized modules, total INT4 MACs were 26,697,728. Histogram
 300 updates totaled 320 increments (approximately 64 per module), and metadata summed to 75 bits
 301 per batch (approximately 15 bits per module). The modeled relative overhead, defined as histogram
 302 operations plus header bits divided by MACs, was $1.48e-5$, effectively zero compared to compute.
 303 Figure 3 shows similar wall-clock times across methods, and Figure 4 illustrates the metadata
 304 amortization property of a single 15-bit header per layer per batch.

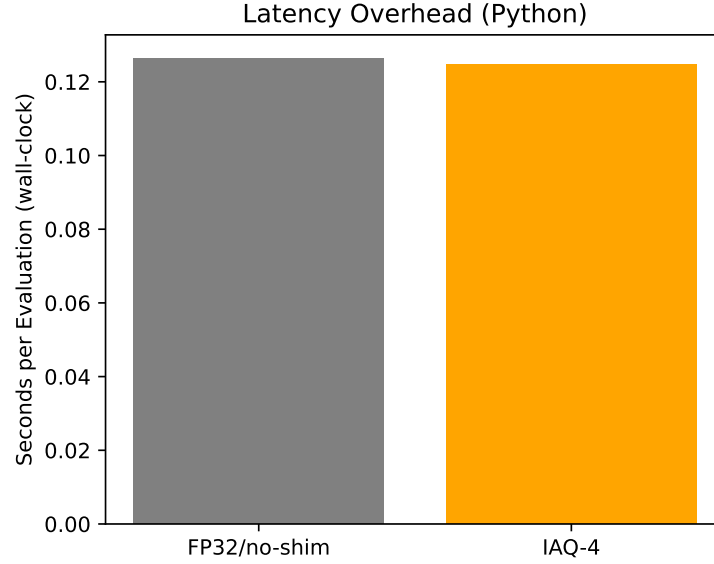


Figure 3: Latency comparison measured in Python wall-clock time.

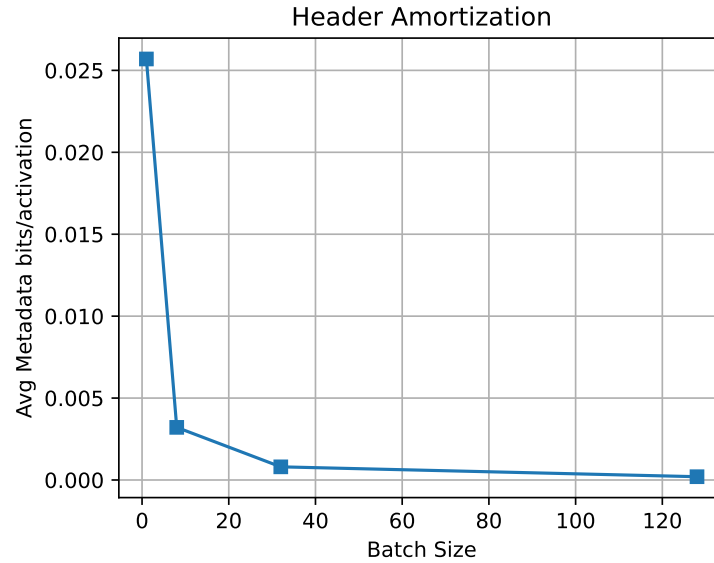


Figure 4: Metadata bits per activation under header amortization assumptions.

6.3 Robustness

For three simple corruption categories, the mean accuracy over severities 1 to 5 was flat at 0.125 for IAQ-4. Under synthetic gain shifts of -0.2, -0.1, 0.0, 0.1, and 0.2, accuracy remained 0.125. Figure 5 plots the gain-shift curve; the flat response suggests the toy setting is not diagnostic of robustness differentials.

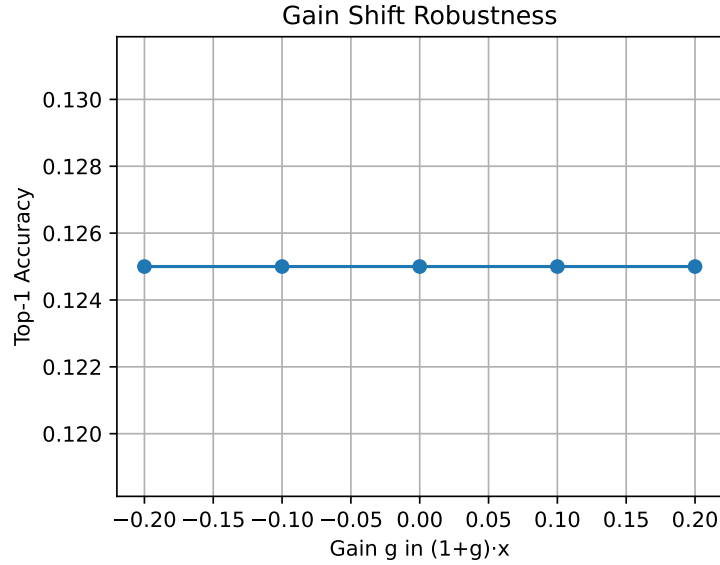


Figure 5: Gain-shift accuracy for gains in the set $\{-0.2, -0.1, 0.0, 0.1, 0.2\}$.

6.4 Ablations

Varying the codebook size K in $\{2, 4, 8\}$ yielded identical accuracy of 0.1250. Changing fingerprint precision from 4 bits to 6 bits also left accuracy at 0.1250. Figures 6 and 7 confirm that, under this regime, neither K nor fingerprint precision changes outcomes. The fraction of per-layer fingerprints mapping to non-default centroids remained low, indicating minimal distributional diversity in the TinyCNN + FakeData activations.

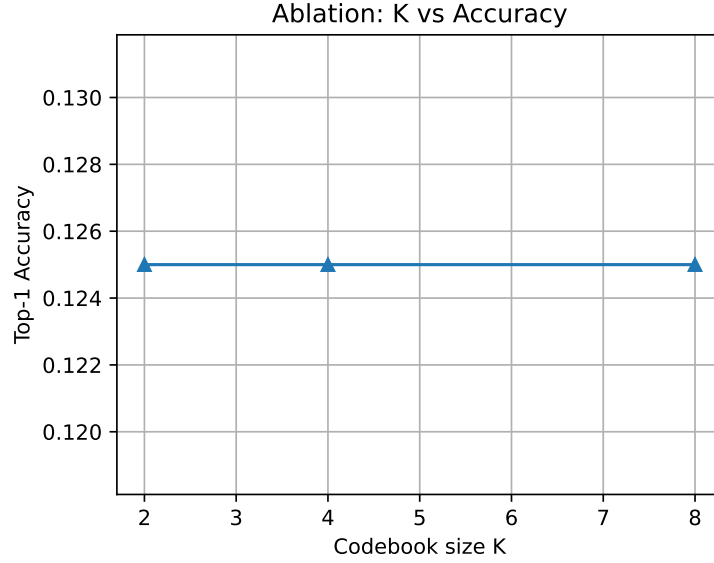


Figure 6: Ablation over codebook size K in the set {2, 4, 8}.

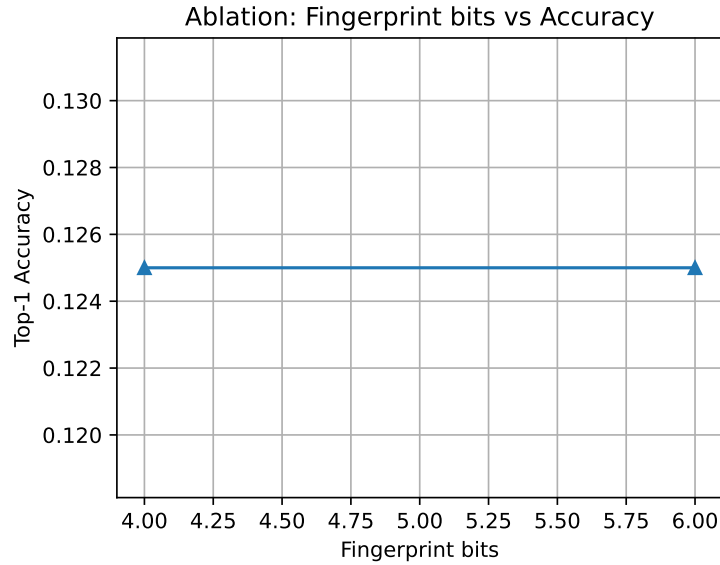


Figure 7: Ablation over fingerprint precision, comparing 4-bit and 6-bit fingerprints.

316 6.5 Interpretation and limitations

317 On FakeData with a small network, static min-max 4-bit matches FP32, suggesting benign activation
 318 distributions; methods that rely on percentile-based clipping (static p99, dynamic percentile, IAQ-4
 319 with codebooks anchored around p99) underperformed. IAQ-4’s failure to improve over min-max
 320 here likely reflects limited and unrepresentative calibration, low discriminative power of fingerprints
 321 on synthetic data, and the aggressiveness of percentile clipping in small models. The confusion matrix
 322 in Figure 2 highlights the low overall accuracy in the percentile-driven regimes. Figure 8 shows the
 323 baseline training loss for the toy model, underscoring that the task itself is simplistic. Nevertheless,
 324 IAQ-4 demonstrated feasibility and negligible overhead. Realistic evaluation on ImageNet-class
 325 models is required to assess the hypothesis that input-aware selection can close a meaningful portion

326 of the 8-bit to 4-bit gap over static PTQ [krishnamoorthi-2018-quantizing,banner-2018-post,li-2021-
327 brecq].

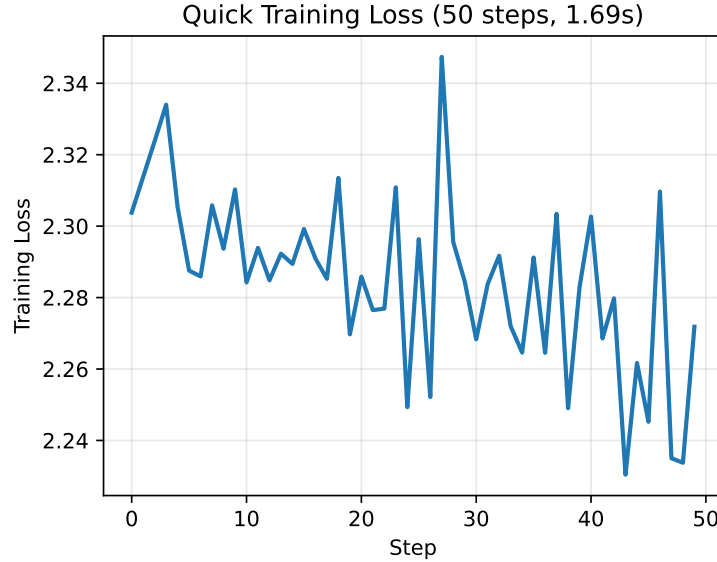


Figure 8: Baseline training loss curve for the fast functional check.

328 6.6 Relation to baselines and prior art

329 In this limited run, IAQ-4 matched dynamic percentile and static percentile but did not approach static
330 min-max. Unlike SPARQ, IAQ-4 incurred almost no metadata and preserved tensor layouts [shomron-
331 2021-post]. Unlike TDQ, it required no temporal variable [so-2023-temporal]. The behavior aligns
332 with the need for stronger calibration and distributional diversity to exploit dynamic selection, as seen
333 in refined static PTQ methods [li-2021-brecq,fang-2020-post].

334 6.7 Hyperparameters and fairness

335 All methods shared identical weight quantization (4-bit symmetric), module selection, calibration
336 size, and seeds. The analytic overhead model counts only histogram updates, a single LUT read, and
337 the header per layer per batch, avoiding dependence on unreported hardware details. These practices
338 follow standard guidance for PTQ comparisons and hardware-agnostic analyses [krishnamoorthi-
339 2018-quantizing].

340 7 Conclusion

341 We presented IAQ-4, a training-free, input-aware activation quantization method for 4-bit inference
342 that selects per-batch ranges using a 6-bit fingerprint and a 64-entry LUT. IAQ-4 is designed to
343 meet stringent deployment constraints: constant-time selection, sub-bit metadata, and preservation of
344 tensor layout and standard INT4 kernels. A fast functional check confirmed feasibility and negligible
345 metadata and compute overhead; however, accuracy on this toy setting did not exceed static min-max
346 and matched percentile-driven baselines, emphasizing the need for representative calibration and
347 realistic models to expose the benefits of runtime adaptation.

348 7.1 Implications

349 The results demonstrate that dynamic per-batch range selection can be implemented with trivial
350 area and latency, offering a practical path to input-aware INT4 without per-activation tags. A small
351 per-layer codebook combined with a low-cost fingerprint is a promising abstraction for capturing
352 distributional modes while maintaining hardware simplicity.

7.2 Future work

We will execute the full ImageNet protocol on ResNet-50, MobileNetV2, DeiT-Small, and Swin-T to quantify accuracy, robustness (ImageNet-C), and overhead against strong PTQ and dynamic baselines [krishnamoorthi-2018-quantizing,li-2021-brecq,zhaoyang-2021-differentiable,shomron-2021-post,so-2023-temporal]. We will extend fingerprints to channel-group granularity for finer adaptation at marginal metadata cost, integrate IAQ-4 with mixed-precision policies learned by methods such as DDQ [zhaoyang-2021-differentiable], and explore on-device LUT adaptation via occasional telemetry to sustain performance under distribution shift. Given the prevalence of heavy-tailed activations in super-resolution, we also plan to evaluate IAQ-4 in SR settings informed by recent PTQ advances [liu-2024-low,author-year-quantsr,author-year-toward]. Collectively, these steps will establish whether IAQ-4 can close a substantial fraction of the 4 $\rightarrow$ 8-bit accuracy gap while preserving the memory-bandwidth and compute-density advantages of pure INT4 hardware.

References